Chapter 13

Logistic regression

Linear regression is an additive model, which does not work for binary outcomes—that is, data y that take on the values 0 or 1. To model binary data, we need to add two features to the base model $y = a + bx$: a nonlinear transformation that bounds the output between 0 and 1 (unlike $a + bx$, which is unbounded), and a model that treats the resulting numbers as probabilities and maps them into random binary outcomes. This chapter and the next describe one such model—logistic regression—and then in Chapter 15 we discuss *generalized linear models*, a larger class that includes linear and logistic regression as special cases. In the present chapter we introduce the mathematics of logistic regression and also its latent-data formulation, in which the binary outcome y is a discretized version of an unobserved or latent continuous measurement z. As with the linear model, we show how to fit logistic regression, interpret its coefficients, and plot data and fitted curves. The nonlinearity of the model increases the challenges of interpretation and model-building, as we discuss in the context of several examples.

13.1 Logistic regression with a single predictor

The *logistic* function,

$$\text{logit}(x) = \log\left(\frac{x}{1-x}\right),$$

maps the range $(0, 1)$ to $(-\infty, \infty)$ and is useful for modeling probabilities. Its inverse function, graphed in Figure 13.1a, maps back to the unit range:

$$\text{logit}^{-1}(x) = \frac{e^x}{1+e^x}. \tag{13.1}$$

In R, we can access these functions using the logistic distribution:

```
logit <- qlogis
invlogit <- plogis
```

We next discuss the use of the logistic transformation in modeling binary data.

Example: modeling political preference given income

Example: Income and voting

Conservative parties have traditionally received more support among voters with higher incomes. We illustrate classical logistic regression with a simple analysis of this pattern from the National Election Study in 1992. For each respondent i in this poll, we label $y_i = 1$ if he or she preferred George Bush (the Republican candidate for president) or 0 if he or she preferred Bill Clinton (the Democratic candidate), excluding respondents who preferred other candidates or had no opinion.[1]

We predict preferences given the respondent's income level, which is characterized on a five-point scale as described on page 149. An alternative would be to use dollar income as a continuous

[1]Data and code for this example are in the folder NES.

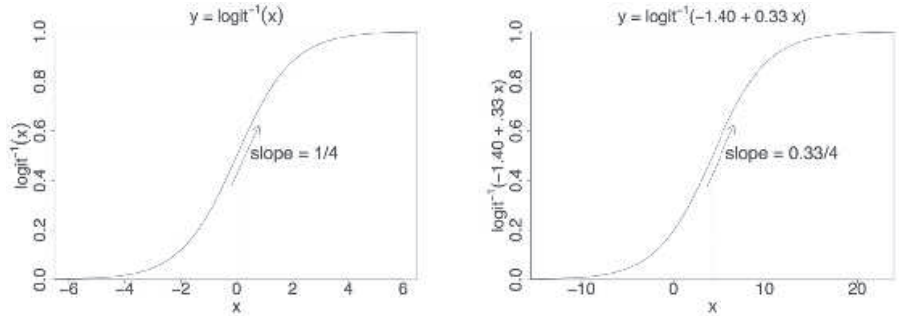


Figure 13.1 *(a) Inverse logit function $logit^{-1}(x)$ from (13.1): the transformation from linear predictors to probabilities that is used in logistic regression. (b) An example of the predicted probabilities from a logistic regression model: $y = logit^{-1}(-1.40 + 0.33x)$. The shape of the curve is the same, but its location and scale have changed; compare the x-axes on the two graphs. On each graph, the vertical dotted line shows where the predicted probability is 0.5: in graph (a), this is at $logit(0.5) = 0$; in graph (b), the halfway point is where $-1.40 + 0.33x = 0$, which is $x = 1.40/0.33 = 4.2$. As discussed in Section 13.2, the slope of the curve at the halfway point is the logistic regression coefficient divided by 4, thus 1/4 for $y = logit^{-1}(x)$ and $0.33/4$ for $y = logit^{-1}(-1.40 + 0.33x)$. The slope of the logistic regression curve is steepest at this halfway point.*

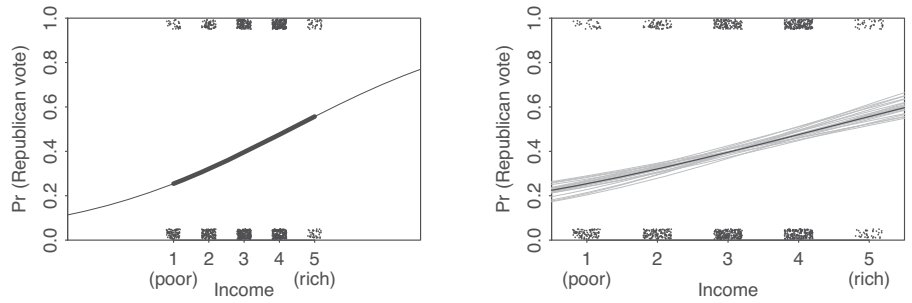


Figure 13.2 *Logistic regression estimating the probability of supporting George Bush in the 1992 presidential election, as a function of discretized income level. Survey data are indicated by jittered dots. In this example, little is revealed by these jittered points, but we want to emphasize here that the data and fitted model can be put on a common scale. (a) Fitted logistic regression: the thick line indicates the curve in the range of the data; the thinner lines at the ends show how the logistic curve approaches 0 and 1 in the limits. (b) In the range of the data, the solid line shows the best-fit logistic regression, and the light lines show uncertainty in the fit.*

predictor, but for predicting partisanship it seems to make more sense to use quantiles, as this captures relative income, which seems more relevant when studying political attitudes. In this case, five income categories were available from the survey response.

The data are shown as (jittered) dots in Figure 13.2, along with the fitted *logistic regression* line, a curve that is constrained to lie between 0 and 1. We interpret the line as the probability that $y = 1$ given x—in mathematical notation, $\Pr(y = 1|x)$.

We fit and display the model using `stan_glm`, specifying logistic regression as follows:

```
fit_1 <- stan_glm(rvote ~ income, family=binomial(link="logit"), data=nes92)
print(fit_1)
```

to yield,

```
            Median MAD_SD
(Intercept) -1.4    0.2
income       0.3    0.1
```

We write the fitted model as $\Pr(y_i = 1) = \text{logit}^{-1}(-1.4 + 0.3 * \text{income})$. We shall define this model mathematically and then return to discuss its interpretation.

Unlike with linear regression, there is no `sigma` in the output. Logistic regression has no separate variance term; its uncertainty comes from its probabilistic prediction of binary outcomes.

Figure 13.2a shows a graph of the data and fitted line, which can be made as follows:

```
plot(nes92$income, nes92$rvote)
curve(invlogit(coef(fit_1)[1] + coef(fit_1)[2]*x), add=TRUE)
```

The R code we actually use to make the figure has more steps so as to display axis labels, jitter the points, adjust line thickness, and so forth.

The logistic regression model

We would run into problems if we tried to fit a linear regression model, $X\beta$ + error, to data y that take on the values 0 and 1. The coefficients in such a model could be interpreted as differences in probabilities, but we could have difficulties using the model for prediction—it would be possible to have predicted probabilities below 0 or above 1—and information would be lost by modeling discrete outcomes as if they were continuous.

Instead, we model the probability that $y = 1$,

$$\Pr(y_i = 1) = \text{logit}^{-1}(X_i\beta), \tag{13.2}$$

under the assumption that the outcomes y_i are independent given these probabilities. We refer to $X\beta$ as the *linear predictor*. We illustrate logistic regression curves schematically in Figure 13.1 and as fitted to the election polling example in Figure 13.2.

Equivalently, model (13.2) can be written,

$$\begin{aligned}\Pr(y_i = 1) &= p_i \\ \text{logit}(p_i) &= X_i\beta,\end{aligned} \tag{13.3}$$

We prefer to work with logit^{-1} as in (13.2) because we find it more natural to focus on the mapping from the linear predictor to the probabilities, rather than the reverse. However, you will need to understand formulation (13.3) to follow the literature.

The inverse logistic function is curved, and so the expected difference in y corresponding to a fixed difference in x is not a constant. As can be seen in Figure 13.1, the steepest change occurs at the middle of the curve. For example:

- $\text{logit}(0.5) = 0$, and $\text{logit}(0.6) = 0.4$. Here, adding 0.4 on the logit scale corresponds to a change from 50% to 60% on the probability scale.
- $\text{logit}(0.9) = 2.2$, and $\text{logit}(0.93) = 2.6$. Here, adding 0.4 on the logit scale corresponds to a change from 90% to 93% on the probability scale.

Similarly, adding 0.4 at the low end of the scale moves a probability from 7% to 10%. In general, any particular change on the logit scale is compressed at the ends of the probability scale, which is needed to keep probabilities bounded between 0 and 1.

Fitting the model using stan_glm and displaying uncertainty in the fitted model

As with linear regression, we can use the `stan_glm` function simulations to work with predictive uncertainty. We can extract random draws of the parameter simulations to display uncertainty in the coefficients by adding the following to the plotting commands from page 219. Here we sample 20 random draws from the simulations of the fitted model, because if we plotted all 4000 the graph would be impossible to read:

```
sims_1 <- as.matrix(fit_1)
n_sims <- nrow(sims_1)
for (j in sample(n_sims, 20)){
  curve(invlogit(sims_1[j,1] + sims_1[j,2]*x), col="gray", lwd=0.5, add=TRUE)
}
```

13.2 Interpreting logistic regression coefficients and the divide-by-4 rule

Coefficients in logistic regression can be challenging to interpret because of the nonlinearity just noted. We shall try to generalize the procedure for understanding coefficients one at a time, as was done for linear regression in Chapter 10. We illustrate with the model, Pr(Bush support) = $\text{logit}^{-1}(-1.40 + 0.33 * \text{income})$. We present some simple approaches here and return in Section 14.4 to more comprehensive numerical summaries.

Evaluation at and near the mean of the data

The nonlinearity of the logistic function requires us to choose where to evaluate changes, if we want to interpret on the probability scale. The average of the predictors in the data can be a useful start.

- As with linear regression, the *intercept* can only be interpreted assuming zero values for the other predictors. When zero is not interesting or not even in the model (as in the voting example, where income is on a 1–5 scale), the intercept must be evaluated at some other point. For example, we can evaluate Pr(Bush support) at the highest income category and get $\text{logit}^{-1}(-1.40+0.33*5) = 0.56$. Or we can evaluate Pr(Bush support) at the mean of respondents' incomes: $\text{logit}^{-1}(-1.40+0.33\,\bar{x})$. In R we can compute,

  ```
  invlogit(-1.40 + 0.33*mean(nes92$income))
  ```

  or, more generally,

  ```
  invlogit(coef(fit_1)[1] + coef(fit_1)[2]*mean(nes92$income))
  ```

  with `invlogit = plogis` as defined on page 217. For this dataset, $\bar{x} = 3.1$, yielding Pr(Bush support) = $\text{logit}^{-1}(-1.40 + 0.33 * 3.1) = 0.41$ at this central point.
- A difference of 1 in income (on this 1–5 scale) corresponds to a positive difference of 0.33 in the logit probability of supporting Bush. We can evaluate how the probability differs with a unit difference in x near the central value. Since $\bar{x} = 3.1$ in this example, we can evaluate the logistic regression function at $x = 3$ and $x = 2$; the difference in $\Pr(y = 1)$ corresponding to adding 1 to x is $\text{logit}^{-1}(-1.40 + 0.33 * 3) - \text{logit}^{-1}(-1.40 + 0.33 * 2) = 0.08$.

The divide-by-4 rule

The logistic curve is steepest at its center, at which point $\alpha + \beta x = 0$ so that $\text{logit}^{-1}(\alpha + \beta x) = 0.5$; see Figure 13.1. The slope of the curve—the derivative of the logistic function—is maximized at this point and attains the value $\beta e^0/(1 + e^0)^2 = \beta/4$. Thus, $\beta/4$ is the maximum difference in $\Pr(y = 1)$ corresponding to a unit difference in x.

As a rule of convenience, we can take logistic regression coefficients (other than the constant term) and divide them by 4 to get an upper bound of the predictive difference corresponding to a unit difference in x. This upper bound is a reasonable approximation near the midpoint of the logistic curve, where probabilities are close to 0.5.

For example, in the model Pr(Bush support) = $\text{logit}^{-1}(-1.40 + 0.33 * \text{income})$, we can divide 0.33 by 4 to get 0.08: a difference of 1 in income category corresponds to no more than an 8% positive difference in the probability of supporting Bush. In this case the probabilities are not far from 50% (see Figure 13.2), and so the divide-by-4 approximation works well.

Interpretation of coefficients as odds ratios

Another way to interpret logistic regression coefficients is in terms of *odds ratios*. If two outcomes have the probabilities $(p, 1-p)$, then $\frac{p}{1-p}$ is called the *odds*. An odds of 1 is equivalent to a probability of 0.5—that is, equally likely outcomes. Odds of 0.5 or 2.0 represent probabilities of $(\frac{1}{3}, \frac{2}{3})$. Dividing

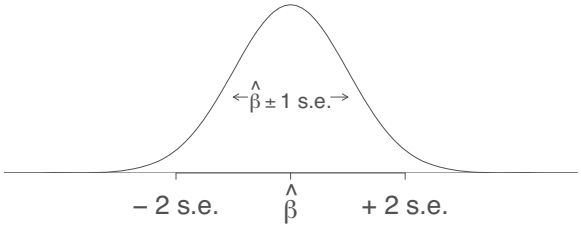


Figure 13.3 *Distribution representing uncertainty in an estimated regression coefficient (repeated from Figure 4.1). The range of this distribution corresponds to the possible values of β that are consistent with the data. When using this as an uncertainty distribution, we assign an approximate 68% chance that β will lie within 1 standard error of the point estimate, $\hat{\beta}$, and an approximate 95% chance that β will lie within 2 standard errors. Assuming the regression model is correct, it should happen only about 5% of the time that the estimate, $\hat{\beta}$, falls more than 2 standard errors away from the true β.*

two odds, $\frac{p_1}{1-p_1} / \frac{p_2}{1-p_2}$, gives an odds ratio. Thus, an odds ratio of 2 corresponds to a change from $p = 0.33$ to $p = 0.5$, or a change from $p = 0.5$ to $p = 0.67$.

An advantage of working with odds ratios (instead of probabilities) is that it is possible to keep scaling up odds ratios indefinitely without running into the boundary points of 0 and 1. For example, going from an odds of 2 to an odds of 4 increases the probability from $\frac{2}{3}$ to $\frac{4}{5}$; doubling the odds again increases the probability to $\frac{8}{9}$, and so forth.

Exponentiated logistic regression coefficients can be interpreted as odds ratios. For simplicity, we illustrate with a model with one predictor, so that

$$\log\left(\frac{\Pr(y=1|x)}{\Pr(y=0|x)}\right) = \alpha + \beta x. \tag{13.4}$$

Adding 1 to x (that is, changing x to $x + 1$ in the above expression) has the effect of adding β to both sides of the equation. Exponentiating both sides, the odds are then multiplied by e^{β}. For example, if $\beta = 0.2$, then a unit difference in x corresponds to a multiplicative change of $e^{0.2} = 1.22$ in the odds (for example, changing the odds from 1 to 1.22, or changing p from 0.5 to 0.55).

We find that the concept of odds can be somewhat difficult to understand, and odds ratios are even more obscure. Therefore we prefer to interpret coefficients on the original scale of the data when possible, for example, saying that adding 0.2 on the logit scale corresponds to a change in probability from $\text{logit}^{-1}(0)$ to $\text{logit}^{-1}(0.2)$.

Coefficient estimates and standard errors

The coefficients in classical logistic regression are estimated using maximum likelihood, a procedure that can often work well for models with few predictors fit to reasonably large samples (but see Section 14.6 for a potential problem).

As with the linear model, the standard errors represent estimation uncertainty. We can roughly say that coefficient estimates within 2 standard errors of $\hat{\beta}$ are consistent with the data. Figure 13.3 shows the normal distribution that approximately represents the range of possible values of β. For the voting example, the coefficient of income has an estimate $\hat{\beta}$ of 0.33 and a standard error of 0.06; thus the data are roughly consistent with values of β in the range $[0.33 \pm 2 * 0.06] = [0.21, 0.45]$.

Statistical significance

As with linear regression, an estimate is conventionally labeled as statistically significant if it is at least 2 standard errors away from zero or, more formally, if its 95% confidence interval excludes zero.

For reasons discussed in earlier chapters, we do *not* recommend using statistical significance to

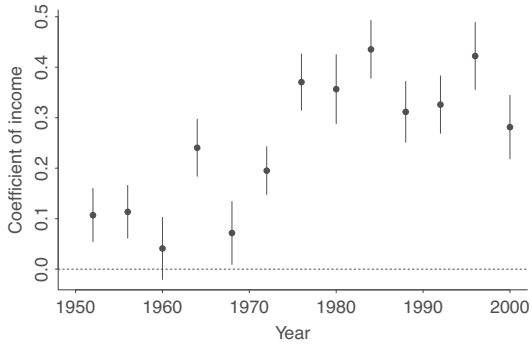


Figure 13.4 *Coefficient of income (on a 1–5 scale) with ±1 standard-error bounds in logistic regressions predicting Republican preference for president, as estimated separately from surveys in the second half of the twentieth century. The pattern of richer voters supporting Republicans has increased since 1970. The data used in the estimate for 1992 appear in Figure 13.2.*

make decisions in logistic regression about whether to include predictors or whether to interpret effects as real. When fitting many models, or when exploring data and deciding on models to fit, it is easy to find patterns—including patterns that are "statistically significant"—arising from chance alone. Conversely, if an estimate is *not* labeled as statistically significant—that is, if it is less than 2 standard errors from zero—there could still be an underlying effect or relationship in the larger population. Thus, we recognize that standard errors convey uncertainty in estimation without trying to make a one-to-one mapping between statistically significant coefficients and real effects.

When considering multiple inputs, we follow the same principles as with linear regression when deciding when and how to include and combine inputs in a model, as discussed in Section 12.6.

Displaying the results of several logistic regressions

Example: Income and voting

We can display estimates from a series of logistic regressions in a single graph, just as was done in Section 10.9 for linear regression coefficients. Figure 13.4 illustrates with the estimate ±1 standard error for the coefficient for income on presidential preference, fit to National Election Studies pre-election polls from 1952 through 2000. Higher income has consistently been predictive of Republican support, but the connection became stronger toward the end of the century.

13.3 Predictions and comparisons

Logistic regression predictions are probabilistic, so for each unobserved future data point y_i^{new}, there is a predictive probability,

$$p_i^{\text{new}} = \Pr(y_i^{\text{new}} = 1) = \text{logit}^{-1}(X_i^{\text{new}}\beta),$$

rather than a point prediction.

We discussed predictions for linear regression in Section 9.2; for logistic regression and other generalized linear models we can similarly get point predictions for the expected value of a new observation using `predict` and then use the functions `posterior_linpred`, `posterior_epred`, and `posterior_predict` to get posterior simulations representing uncertainty about the linear predictor $X^{\text{new}}\beta$, the predicted probability $\text{logit}^{-1}(X^{\text{new}}\beta)$, and the random binary outcome y^{new}, respectively. We demonstrate with a simple example.

Point prediction using `predict`

Earlier we fit a logistic regression to predict the probability that respondents in a survey from 1992 would support George Bush for president. The fitted model is Pr(Bush support) = $\text{logit}^{-1}(-1.40 + 0.33 * \text{income})$, and it is saved as the R object `fit1`, which includes posterior simulations representing uncertainty in the fitted coefficients.

Now suppose we want to predict the vote preference of a person with income level 5 (recall the five-point scale in Figure 13.2). We can use the `predict` function to compute the estimated probability that he or she supports Bush, $\text{E}(y^{\text{new}}|x^{\text{new}}) = \Pr(y^{\text{new}} = 1|x^{\text{new}})$:

```
new <- data.frame(income=5)
pred <- predict(fit_1, type="response", newdata=new)
```

The result is 0.56, a predicted 56% chance of supporting Bush. Alternatively we could directly compute the prediction using the point estimate, `invlogit(-1.40 + 0.33*5)`. Because the logistic transformation is nonlinear, the result from performing `invlogit` on the fitted linear predictor is not quite identical to what is obtained from `predict`, but in this case the difference is tiny: the coefficients are estimated with enough precision that there is very little uncertainty in the linear predictor, and the logistic transformation is approximately linear over any narrow range.

In the above call to `predict`, we specified `type="response"` to get the prediction on the probability scale. Alternatively, we could set `type="link"` to get the estimated linear predictor, the prediction on the logit scale, which in this case is $0.25 = -1.40 + 0.33 * 5$. The command `invlogit(predict(fit_1, type="link", newdata=new))` gives the same result as `predict(fit_1, type="response", newdata=new)`.

Linear predictor with uncertainty using `posterior_linpred`

As with linear regression, the function `posterior_linpred` yields simulation draws for the linear predictor, $X^{\text{new}}\beta$:

```
linpred <- posterior_linpred(fit_1, newdata=new)
```

The result in this case is a vector representing the posterior distribution of $a + b * 5$. Typically, though, we are interested in the predicted value on the probability scale.

Expected outcome with uncertainty using `posterior_epred`

We can compute the uncertainty in the expected value or predicted probability, $\text{E}(y^{\text{new}}) = \Pr(y^{\text{new}} = 1) = \text{logit}^{-1}(X^{\text{new}}\beta)$ using the `posterior_epred` function:

```
epred <- posterior_epred(fit_1, newdata=new)
```

The result is a vector of length equal to the number of posterior simulation draws; we can summarize, for example, using the mean and standard deviation:

```
print(c(mean(epred), sd(epred)))
```

The result is 0.56 (the same as computed using `predict`) and 0.03. According to our fitted model, the percentage of Bush supporters in the population at that time, among people with income level 5, was probably in the range $56\% \pm 3\%$. This inference implicitly assumes that the survey respondents within each income category are a random sample from the population of interest, and that the logistic regression model accurately describes the association between income and vote preference.

The command `invlogit(posterior_linpred(fit_1, newdata=new))` gives the same result as `posterior_epred(fit_1, newdata=new)`, just converting from the linear predictor to the probability scale.

Predictive distribution for a new observation using `posterior_predict`

To get the uncertainty for a single voter corresponding to some individual data point x^{new}, we use the posterior prediction:

```
postpred <- posterior_predict(fit_1, newdata=new)
```

which returns a vector of length `n_sims`, taking on values 0, 1, 1, 0, 1, ..., corresponding to possible values of `vote` among people with `income=5`. Taking the average of these simulations, `mean(postpred)`, gives 0.56, as this is equivalent to computing the point prediction, $\text{E}(y^{\text{new}}|x^{\text{new}})$, for this new data point.

Prediction given a range of input values

Above we used the prediction functions for a single new data point. We can also use them to make predictions for a vector of new observations. For example, suppose we want to make predictions for five new people whose incomes take on the values 1 through 5:

```
new <- data.frame(income=1:5)
pred <- predict(fit_1, type="response", newdata=new)
linpred <- posterior_linpred(fit_1, newdata=new)
epred <- posterior_epred(fit_1, newdata=new)
postpred <- posterior_predict(fit_1, newdata=new)
```

The first two objects above are vectors of length 5; the last three are `n_sims` $\times$ 5 matrices.

We can use `epred` to make statements about the population. For example, this will compute the posterior probability, according to the fitted model, that Bush was more popular among people with income level 5 than among people with income level 4:

```
mean(epred[,5] > epred[,4])
```

And this will compute a 95% posterior distribution for the difference in support for Bush, comparing people in the richest to the second-richest category:

```
quantile(epred[,5] - epred[,4], c(0.025, 0.975))
```

We can use `postpred` to make statements about individual people. For example, this will compute the posterior simulations of the number of these new survey respondents who support Bush:

```
total <- apply(postpred, 1, sum)
```

And here is how to compute the probability that at least three of them support Bush:

```
mean(total >= 3)
```

Logistic regression with just an intercept

As discussed in Section 7.3, linear regression with just an intercept is the same as estimating an average, and linear regression with a single binary predictor is the same as estimating a difference in averages. Similarly, logistic regression with just an intercept is equivalent to the estimate of a proportion.

Here is an example. A random sample of 50 people are tested, and 10 have a particular disease. The proportion is 0.20 with standard error $\sqrt{0.2 * 0.8/50} = 0.06$. Alternatively we can set this up as logistic regression:

```
y <- rep(c(0, 1), c(40, 10))
simple <- data.frame(y)
fit <- stan_glm(y ~ 1, family=binomial(link="logit"), data=simple)
```

yielding the result,

```
             Median MAD_SD
(Intercept) -1.38   0.35
```

We can transform the prediction to the probability scale and get $\text{logit}^{-1}(-1.38) = 0.20$, as with the direct calculation, and then obtain approximate ± 1 standard error bounds using $\text{logit}^{-1}(-1.38 \pm 0.35) = (\text{logit}^{-1}(1.73), \text{logit}^{-1}(1.03)) = (0.15, 0.26)$, which is essentially the same as the classical estimate with uncertainty of 0.20 ± 0.06.

Alternatively, we can get the inference on the probability scale using `posterior_epred`:

```
new <- data.frame(x=0)
epred <- posterior_epred(fit, newdata=new)
print(c(mean(epred), sd(epred)))
```

This again returns 0.20 and 0.06. If you go to more decimal places, the classical and logistic regression estimates differ, in part because `stan_glm` uses a prior distribution (see Section 13.5) and in part because the classical standard error is only an approximation to the inferential uncertainty arising from discrete data.

Data on the boundary. What happens when $y = 0$ or n? As discussed on page 52, the classical standard error formula gives 0 at these extremes, so instead we use the 95% interval $\hat{p} \pm \sqrt{\hat{p}(1-\hat{p})/(n+4)}$, where $\hat{p} = (y+2)/(n+4)$. Let's compare this to logistic regression, using the example of a test with zero positive results out of 50:

```
y <- rep(c(0, 1), c(50, 0))
simple <- data.frame(y)
fit <- stan_glm(y ~ 1, family=binomial(link="logit"), data=simple)
```

Here is the result:

```
             Median MAD_SD
(Intercept) -4.58   1.14
```

The approximate 95% interval is $\text{logit}^{-1}(-4.58 \pm 2 * 1.14) = (0, 0.09)$. The lower endpoint is not exactly zero but we bring it down to zero in any case, as the data are obviously consistent with an underlying rate of zero. In this case, the classical $\hat{p} \pm \sqrt{\hat{p}(1-\hat{p})/(n+4)}$ interval is also $(0, 0.09)$ to two decimal places. Again, the intervals based on the simple formula and logistic regression will not be identical, as the two approaches make different assumptions.

Logistic regression with a single binary predictor

Logistic regression on an indicator variable is equivalent to a comparison of proportions. For a simple example, consider tests for a disease on samples from two different populations, where 10 out of 50 from population A test positive, as compared to 20 out of 60 from population B. The classical estimate is 0.13 with standard error 0.08. Here is the setup as logistic regression:

```
x <- rep(c(0, 1), c(50, 60))
y <- rep(c(0, 1, 0, 1), c(40, 10, 40, 20))
simple <- data.frame(x, y)
fit <- stan_glm(y ~ x, family=binomial(link="logit"), data=simple)
```

which yields:

```
             Median MAD_SD
(Intercept) -1.39   0.35
x            0.69   0.44
```

To get inference for the difference in probabilities, we compare predictions on the probability scale for $x = 0$ and $x = 1$:

```
new <- data.frame(x=c(0, 1))
epred <- posterior_epred(fit, newdata=new)
diff <- epred[,2] - epred[,1]
print(c(mean(diff), sd(diff)))
```

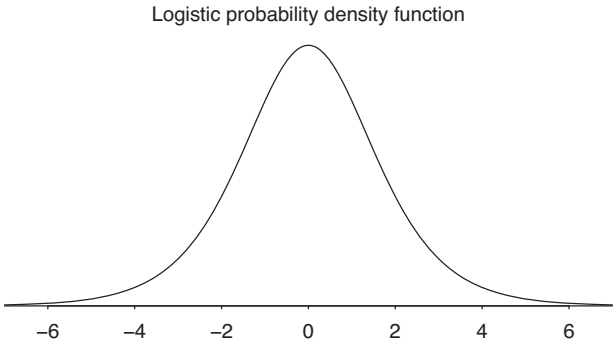


Figure 13.5 *The probability density function of the logistic distribution, which is used for the error term in the latent-data formulation (13.5) of logistic regression. The logistic curve in Figure 13.1a is the cumulative distribution function of this density. The maximum of the density is 0.25, which corresponds to the maximum slope of 0.25 in the inverse logit function of Figure 13.1a.*

This returns 0.13 and 0.08, the same estimate and standard error as computed using the simple formula. The point of this example is not that you should use logistic regression to do simple comparisons of proportions; rather, we are demonstrating the underlying unity of comparisons and logistic regression for binary data, in the same way that Section 7.3 shows how comparisons of continuous data is a special case of linear regression.

13.4 Latent-data formulation

We can interpret logistic regression directly—as a nonlinear model for the probability of a "success" or "yes" response given some predictors—and also indirectly, using what are called unobserved or *latent* variables.

In this formulation, each discrete outcome y_i is associated with a continuous, unobserved outcome z_i, defined as follows:

$$y_i = \begin{cases} 1 & \text{if } z_i > 0 \\ 0 & \text{if } z_i < 0 \end{cases}$$
$$z_i = X_i\beta + \epsilon_i, \tag{13.5}$$

with independent errors ϵ_i that have the *logistic* probability distribution. The logistic distribution is shown in Figure 13.5 and is defined so that

$$\Pr(\epsilon_i < x) = \text{logit}^{-1}(x) \text{ for all } x.$$

Thus, $\Pr(y_i = 1) = \Pr(z_i > 0) = \Pr(\epsilon_i > -X_i\beta) = \text{logit}^{-1}(X_i\beta)$, and so models (13.2) and (13.5) are equivalent.

Figure 13.6 illustrates for an observation i with income level $x_i = 1$ (that is, a person in the lowest income category), whose linear predictor, $X_i\beta$, thus has the value $-1.40 + 0.33 * 1 = -1.07$. The curve illustrates the distribution of the latent variable z_i, and the shaded area corresponds to the probability that $z_i > 0$, so that $y_i = 1$. In this example, $\Pr(y_i = 1) = \text{logit}^{-1}(-1.07) = 0.26$.

Interpretation of the latent variables

Latent variables are a computational trick but they can also be interpreted substantively. For example, in the pre-election survey, $y_i = 1$ for Bush supporters and 0 for Clinton supporters. The unobserved

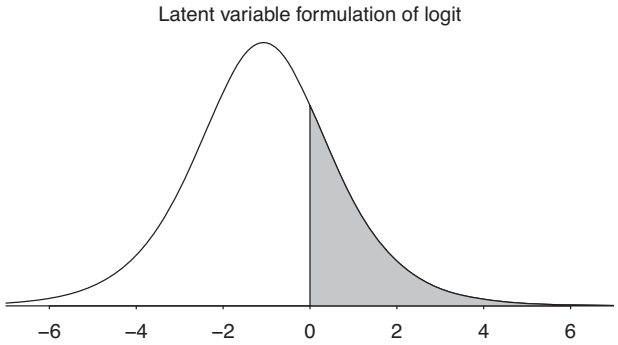


Figure 13.6 *The probability density function of the latent variable z_i in model (13.5) if the linear predictor, $X_i\beta$, has the value −1.07. The shaded area indicates the probability that $z_i > 0$, so that $y_i = 1$ in the logistic regression.*

continuous z_i can be interpreted as the respondent's "utility" or preference for Bush, compared to Clinton: the sign of the utility tells us which candidate is preferred, and its magnitude reveals the strength of the preference.

Only the sign of z_i, not its magnitude, can be determined directly from binary data. However, we can learn more about the z_i's given the logistic regression predictors. In addition, in some settings direct information is available about the z_i's; for example, a survey can ask "feeling thermometer" questions such as, "Rate your feelings about Barack Obama on a 1–10 scale, with 1 being the most negative and 10 being the most positive."

Nonidentifiability of the latent scale parameter

In Section 14.6 we discuss challenges of identification, situations where the data do not supply precise information about parameters in a model, a concept introduced earlier on page 146 in Section 10.7. A particular identification issue arises with the scaling of the latent model in logistic regression.

The logistic probability density function in Figure 13.5 appears bell-shaped, much like the normal density that is used for errors in linear regression. In fact, the logistic distribution is very close to the normal distribution with mean 0 and standard deviation 1.6—an identity that we discuss further on page 272 in the context of "probit regression." For now, we merely note that the logistic model (13.5) for the latent variable z is closely approximated by the normal regression model,

$$z_i = X_i\beta + \epsilon_i, \quad \epsilon_i \sim \text{N}(0, \sigma^2), \tag{13.6}$$

with $\sigma = 1.6$. This then raises the question, Why not estimate σ?

We cannot estimate the parameter σ in model (13.6) because it is not identified when considered jointly with β. If all the elements of β are multiplied by a positive constant and σ is also multiplied by that constant, then the model does not change. For example, suppose we fit the model

$$z_i = -1.40 + 0.33x_i + \epsilon_i, \quad \epsilon_i \sim \text{N}(0, 1.6^2).$$

This is equivalent to the model

$$z_i = -14.0 + 3.3x_i + \epsilon_i, \quad \epsilon_i \sim \text{N}(0, 16^2),$$

or

$$z_i = -140 + 33x_i + \epsilon_i, \quad \epsilon_i \sim \text{N}(0, 160^2).$$

As we move from each of these models to the next, z is multiplied by 10, but the *sign* of z does not change. Thus all the models have the same implications for the observed data y: for each model,

$\Pr(y_i = 1) \approx \text{logit}^{-1}(-1.40 + 0.33x_i)$. This is only approximate because the logistic distribution is not exactly normal.

Thus, model (13.6) has an essential indeterminacy when fit to binary data, and it is standard to resolve this by setting the variance parameter σ to a fixed value such as 1.6, chosen because the normal distribution with standard deviation 1.6 is close to the unit logistic distribution.

13.5 Maximum likelihood and Bayesian inference for logistic regression

Classical linear regression can be motivated in a purely algorithmic fashion (as least squares) or as maximum likelihood inference under a normal model; see Section 10.7. With logistic regression (and other generalized linear models, as described in the next chapter), the usual practice is to start with maximum likelihood, followed by Bayesian inference and other regularizing techniques.

Maximum likelihood using iteratively weighted least squares

For binary logistic regression with data $y_i = 0$ or 1, the likelihood is

$$p(y|\beta, X) = \prod_{i=1}^{n} \begin{cases} \text{logit}^{-1}(X_i\beta) & \text{if } y_i = 1 \\ 1 - \text{logit}^{-1}(X_i\beta) & \text{if } y_i = 0, \end{cases}$$

which can be written more compactly, but equivalently, as

$$p(y|\beta, X) = \prod_{i=1}^{n} \left(\text{logit}^{-1}(X_i\beta)\right)^{y_i} \left(1 - \text{logit}^{-1}(X_i\beta)\right)^{1-y_i}. \tag{13.7}$$

To find the β that maximizes this expression, one can compute the derivative $dp(y|\beta, X)/d\beta$ of the likelihood (or, more conveniently, the derivative of the logarithm of the likelihood), set this derivative equal to 0, and solve for β. There is no closed-form solution, but the maximum likelihood estimate can be found using an iterative optimization algorithm that converges to a point of zero derivative and thus the vector of coefficients β that maximizes (13.7), when such a maximum exists. We shall not discuss this further here, as these computations are all now done invisibly by the computer. We mention it only to give a sense of how likelihood functions are used in classical estimation. Section 14.6 discusses collinearity and separation, which are two conditions under which no maximum likelihood estimate exists.

Bayesian inference with a uniform prior distribution

If the prior distribution on the parameters is uniform, then the posterior density is proportional to the likelihood function, and the posterior mode—the vector of coefficients β that maximizes the posterior density—is the same as the maximum likelihood estimate.

As with linear regression, the benefit of Bayesian inference with a noninformative prior is that we can use simulations from the entire posterior distribution—not just a maximum or any other point estimate—to summarize uncertainty, and we can also use these simulations to make probabilistic predictions. In `stan_glm` we can declare a uniform prior by setting `prior=NULL, prior_intercept=NULL`. We do not generally recommend that choice, as this can result in having no estimate: as we discuss in the Section 14.6, there are real-world logistic regressions in which the coefficients are not identified from data alone, and prior information is required to obtain any estimates at all.

Default prior in `stan_glm`

By default, `stan_glm` uses a weakly informative family of prior distributions. For a logistic regression of the form, $\Pr(y = 1) = \text{logit}^{-1}(a + b_1x_1 + b_2x_2 + \cdots + b_Kx_K)$, each coefficient b_k is given a

normal prior distribution with mean 0 and standard deviation $2.5/\text{sd}(x_k)$. The intercept a is not given a prior distribution directly; instead we assign the prior to the linear predictor with each x set to its mean value in the data, that is, $a + b_1\bar{x}_1 + b_2\bar{x}_2 + \cdots + b_K\bar{x}_K$. This centered intercept is given a normal prior distribution with mean 0 and standard deviation 2.5. This is similar to the default priors for linear regression discussed on page 124, except that for logistic regression there is no need to scale based on the outcome variable.

Bayesian inference with some prior information

When prior information is available, it can be used to increase the precision of estimates and predictions. We demonstrate with a simple hypothetical example of a logistic regression with one predictor, $\Pr(y_i = 1) = \text{logit}^{-1}(a + bx)$, in some area of application for which we expect the slope b to fall between 0 and 1. We assume this is a "soft constraint," not a "hard constraint"; that is, we expect the value of b for any given problem in this domain to be between 0 and 1, but with it being possible on occasion for b to fall outside that range. We shall summarize this soft constraint as a normal prior distribution on b with mean 0.5 and standard deviation 0.5, which implies that, for any given problem, b has a 68% chance of being between 0 and 1 and a 95% chance of being between -0.5 and 1.5. We also include by default a weak prior for the intercept.

Comparing maximum likelihood and Bayesian inference using a simulation study

Example: Prior and data for logistic regression

To understand the role of the prior distribution, we consider several data scenarios, each time assuming that the true parameter values are $a = -2$ and $b = 0.8$ and that data are gathered from an experiment where the values of x are drawn from a uniform distribution between -1 and 1. These choices are entirely arbitrary, just to have a specific model for our simulation study. Here is the R code:[2]

```
library("arm", "rstanarm")
bayes_sim <- function(n, a=-2, b=0.8){
  x <- runif(n, -1, 1)
  z <- rlogis(n, a + b*x, 1)
  y <- ifelse(z>0, 1, 0)
  fake <- data.frame(x, y, z)
  glm_fit <- glm(y ~ x, family=binomial(link="logit"), data=fake)
  stan_glm_fit <- stan_glm(y ~ x, family=binomial(link="logit"), data=fake,
    prior=normal(0.5, 0.5))
  display(glm_fit, digits=1)
  print(stan_glm_fit, digits=1)
}
```

We set `digits=1` in the displays to avoid being distracted by details.

We then simulate for a range of sample sizes, each time focusing on inference about b:

- $n = 0$. With no new data, the above function does not run (as it would have to work with empty data vectors), but in any case the posterior is the same as the prior, centered at 0.5 with standard deviation 0.5.
- $n = 10$. With only 10 observations, the maximum likelihood estimate is noisy, and we expect the Bayesian posterior to be close to the prior. Here's an example of what happens after typing `bayes_sim(10)` in the R console:

```
glm:        coef.est coef.se
(Intercept) -4.4      2.9
x            5.3      4.4
---
stan_glm:   Median MAD_SD
```

[2]Code for this example is in the folder `LogisticPriors`.

```
(Intercept) -1.8    0.8
x            0.6    0.5
```

We shall focus on the coefficient for x, which represents the parameter of interest in this hypothetical study. In the above simulation, `glm` gives a maximum likelihood estimate of 5.3, which is far from the specified belief that b is likely to be in the range $(0, 1)$—but that estimate also has a large standard error, indicating that the likelihood provides little information in this $n = 10$ setting. In contrast, the inference from `stan_glm` relies heavily on the prior distribution: the Bayes estimate of 0.6 is close to the prior mean of 0.5, being pulled away from the data only slightly.

- $n = 100$. Next we try `bayes_sim(100)`, which yields,

```
glm:        coef.est coef.se
(Intercept) -2.6      0.5
x            1.4      0.7
---
stan_glm:   Median MAD_SD
(Intercept) -2.4    0.4
x            0.8    0.4
```

The maximum likelihood estimate is again extreme, but less so than before, and the Bayes estimate is again pulled toward the prior mean of 0.5, but less so than before. This is just one realization of the process, and another random simulation will give different results—try it yourself!—but it illustrates the general pattern of the Bayesian posterior estimate being a compromise between data and prior.

- $n = 1000$. Finally, we simulate an experiment with 1000 data points. Here the sample size is large enough that we expect the likelihood to dominate. Let's look at a simulation:

```
glm:        coef.est coef.se
(Intercept) -2.0      0.1
x            1.1      0.2
---
stan_glm:   Median MAD_SD
(Intercept) -2.0    0.1
x            1.0    0.2
```

The Bayes estimate is only very slightly pooled toward the prior mean. In this particular example, once n is as large as 1000, the prior distribution doesn't really make a difference.

13.6 Cross validation and log score for logistic regression

When evaluating predictive and generalization performance, we could compare binary observations to binary predictions and compute, for example, the percentage of correct predictions, but that completely ignores the additional information in the predicted probabilities. Probability forecasts are sometimes evaluated using the Brier score, which is defined as $\frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2$; this is equivalent to the mean of squared residuals and is proportional to the posterior residual standard deviation discussed in Section 11.6. When computing Bayesian R^2 for linear regression, the posterior residual standard deviation is replaced with the modeled σ, and correspondingly for logistic regression we can use the model-based approximation, $\frac{1}{n}\sum_{i=1}^{n} p_i(1 - p_i)$. However, R^2 for logistic regression has shortcomings, as the residuals are not additive and thus it is not as interpretable as for linear regression. In Section 11.8 we discussed summarizing prediction errors by the log score, which is also a natural choice when the predictions are probabilities, as with logistic regression. Sometimes *deviance*, which is -2 times the log score, is reported.

As discussed in Section 11.8 in the context of linear regression, we can use cross validation to avoid the overfitting associated with using the same data for model fitting and evaluation. Thus, rather than evaluating log predictive probabilities directly, we can compute leave-one-out (LOO) log

prediction probabilities, which can be used to compute the LOO log score (or, if desired, the LOO Brier score, LOO R^2, or any other measure of fit).

Understanding the log score for discrete predictions

We develop intuition about the logarithms of the modeled probability of the observed outcomes using a simple example.

Consider the following scenario: We fit a logistic regression model with estimated parameters $\hat{\beta}$ and apply it to n^{new} new data points with predictor matrix X^{new}, thus yielding a vector of predictive probabilities $p_i^{\text{new}} = \text{logit}^{-1}(X_i^{\text{new}}\hat{\beta})$. We then find out the outcomes y^{new} for the new cases and evaluate the model fit using

$$\text{out-of-sample log score} = \sum_{i=1}^{n^{\text{new}}} \begin{cases} \log p_i^{\text{new}} & \text{if } y_i^{\text{new}} = 1 \\ \log(1 - p_i^{\text{new}}) & \text{if } y_i^{\text{new}} = 0. \end{cases}$$

The same idea applies more generally to any statistical model: the predictive log score is the sum of the logarithms of the predicted probabilities of each data point.

Probabilities are between 0 and 1, and the logarithm of any number less than 1 is negative, with $\log(1) = 0$. Thus the log score is necessarily negative (except in the trivial case of a model that predicts events with perfect probabilities of 1). In general, higher values of log score are better. For example, if two models are used to predict the same data, a log score of -150 is better than a log score of -160.

How large should we expect a log score to be? We can calibrate by considering a model with no information that assigns 50% probability to every outcome. When $p_i^{\text{new}} = 0.5$, the log score equals $\log(0.5) = -0.693$ for every data point y_i^{new}, whether 0 or 1, and thus the log score for n^{new} new data points is simply $-0.693\, n^{\text{new}}$.

What about something better? Consider a model that predicts $p_i = 0.8$ for half the cases and $p_i = 0.6$ for the other half. (This could be obtained, for example, from the model, $\Pr(y_i = 1) = \text{logit}^{-1}(0.4 + x_i)$, for a dataset in which half the cases have $x_i = 1$ and half have $x_i = 0$.) Further suppose that this model has been fit to data that are representative of the general population, so that when we apply it to new cases it will be *calibrated*: when $p_i^{\text{new}} = 0.8$, the new observation y_i^{new} will equal 1 in 80% of the cases; and when $p_i^{\text{new}} = 0.6$, the new observation y_i^{new} will equal 1 in 60% of the cases. In that case, the expected value of the log score for a single new observation chosen at random will be $0.5 * (0.8 \log 0.8 + 0.2 \log 0.2) + 0.5 * (0.6 \log 0.6 + 0.4 \log 0.4) = -0.587$, and so we would expect the log score for a new dataset to be approximately $-0.587\, n^{\text{new}}$.

In this simple example, when we move from the simple coin-flip model to the model with probabilities 0.8 and 0.6, the log score improves by $0.693\, n^{\text{new}} - 0.587\, n^{\text{new}} = 0.106\, n^{\text{new}}$, an improvement of about 1 in the log score per 10 data points.

Log score for logistic regression

Example: Income and voting

We explore the log score using the example of the model in Section 13.1 of polling during the 1992 U.S. presidential election campaign. The survey we are analyzing had 702 respondents who expressed support for Bill Clinton and 477 who preferred George Bush. We compute the log score for a sequence of models using increasing amounts of information:

- The null model gives a 0.5 probability of each outcome; the log score for our 1179 survey respondents is is $1179 \log(0.5) = -817$.
- Just using the fact that 59.5% of survey respondents supported Clinton and 40.5% supported Bush, we can assign 40.5% to $\Pr(y_i = 1)$ for each respondent, which improves the log score to $477 \log(0.405) + 702 \log(-.595) = -796$. Equivalently, we can compute this by fitting a logistic regression with only the constant term:

```
fit_1a <- stan_glm(rvote ~ 1, family=binomial(link="logit"), data=nes92)
```

and then summing the logarithms of the predicted probabilities:

```
predp_1a <- predict(fit_1a, type="response")
y <- nes92$rvote
logscore_1a <- sum(y*log(predp_1a) + (1-y)*log(1 - predp_1a))
```

The regression yields an estimate of the constant term of -0.383, which is the logit of the predicted probability, 0.405, and the log score is -796, as above.

- The fit further improves when we include income as a predictor:

```
fit_1 <- stan_glm(rvote ~ income, family=binomial(link="logit"), data=nes92)
predp_1 <- predict(fit_1, type="response")
logscore_1 <- sum(y*log(predp_1) + (1-y)*log(1 - predp_1))
```

The new log score is -778.

Dividing these log scores by the sample size, we can summarize as follows: using a constant probability estimated from the data improves the log score per observation by $(817-796)/1179 = 0.018$ compared to the null ($p = 0.5$) model, and including income as a predictor increases the log score per observation by a further $(796 - 778)/1179 = 0.015$.

In Section 11.7 we discussed how within-sample validation is optimistic and how cross validation can be used to approximate external validation. Running leave-one-out cross validation,

```
loo(fit_1)
```

gives output,

```
          Estimate   SE
elpd_loo   -780.4  8.6
p_loo         2.0  0.1
looic      1560.9 17.2
```

The LOO estimated log score (`elpd_loo`) of -780 is 2 lower than the within-sample log score of -778 computed above; this difference is about what we would expect, given that the fitted model has 2 parameters or degrees of freedom.

13.7 Building a logistic regression model: wells in Bangladesh

Example: Arsenic in Bangladesh

We illustrate the steps of building, understanding, and checking the fit of a logistic regression model using an example from economics (or perhaps it is psychology or public health): modeling the decisions of households in Bangladesh about whether to change their source of drinking water.[3]

Background

Many of the wells used for drinking water in Bangladesh and other South Asian countries are contaminated with natural arsenic, affecting an estimated 100 million people. Arsenic is a cumulative poison, and exposure increases the risk of cancer and other diseases, with risks estimated to be proportional to exposure.

Any locality can include wells with a range of arsenic levels, as can be seen from the map in Figure 13.7 of all the wells in a collection of villages in a small area of Bangladesh. The bad news is that even if your neighbor's well is safe, it does not mean that yours is also safe. However, the corresponding good news is that, if your well has a high arsenic level, you can probably find a safe well nearby to get your water from—if you are willing to walk the distance and your neighbor is willing to share. (The amount of water needed for drinking is low enough that adding users to a

[3]Data and code for this example are in the folder `Arsenic`.

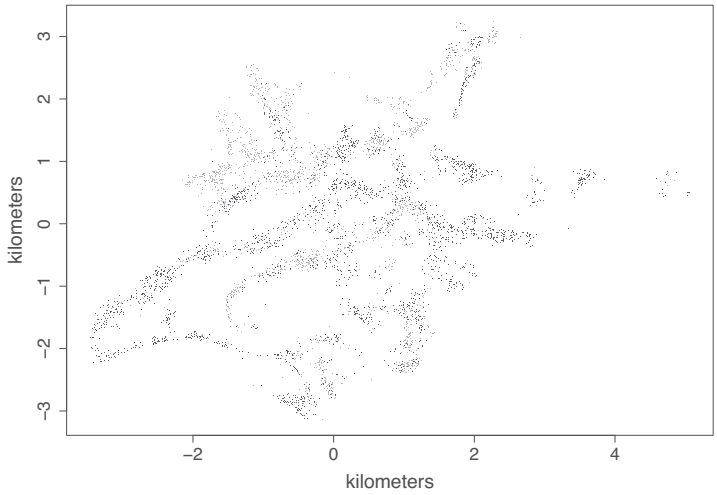


Figure 13.7 *Wells in an area of Araihazar, Bangladesh. Light and dark dots represent wells with arsenic greater than and less than the safety standard of 0.5 (in units of hundreds of micrograms per liter). The wells are located where people live. The empty areas between the wells are mostly cropland. Safe and unsafe wells are intermingled in most of the area, which suggests that users of unsafe wells can switch to nearby safe wells.*

well would not exhaust its capacity, and the surface water in this area is subject to contamination by microbes, hence the desire to use water from deep wells.)

In the area shown in Figure 13.7, a research team from the United States and Bangladesh measured all the wells and labeled them with their arsenic level as well as a characterization as "safe" (below 0.5 in units of hundreds of micrograms per liter, the Bangladesh standard for arsenic in drinking water) or "unsafe" (above 0.5). People with unsafe wells were encouraged to switch to nearby private or community wells or to new wells of their own construction.

A few years later, the researchers returned to find out who had switched wells. We shall perform a logistic regression analysis to understand the factors predictive of well switching among the users of unsafe wells. In the notation of the previous section, our outcome variable is

$$y_i = \begin{cases} 1 & \text{if household } i \text{ switched to a new well} \\ 0 & \text{if household } i \text{ continued using its own well.} \end{cases}$$

We consider the following inputs:

- The distance (in meters) to the closest known safe well.
- The arsenic level of respondent's well.
- Whether any members of the household are active in community organizations.
- The education level of the head of household.

We shall first fit the model just using distance to the nearest well and then put in arsenic concentration, organizational membership, and education.

Logistic regression with just one predictor

We fit the logistic regression in R:

```
fit_1 <- stan_glm(switch ~ dist, family=binomial(link="logit"), data=wells)
```

Displaying this yields,

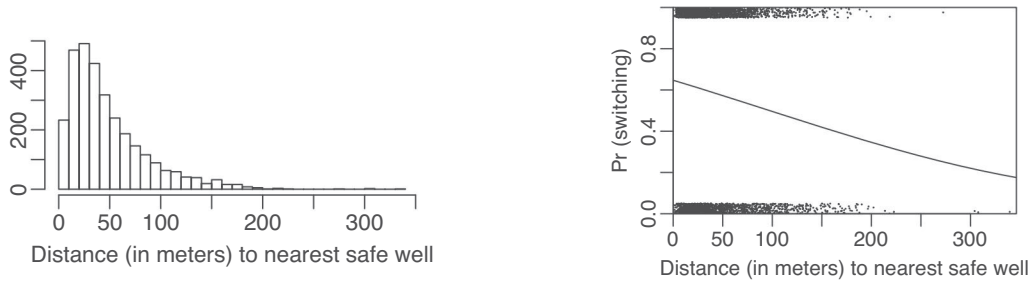


Figure 13.8 *(a) Histogram of distance to the nearest safe well, for each of the unsafe wells in our data from Araihazar, Bangladesh (see Figure 13.7). (b) Graphical expression of the fitted logistic regression, Pr (switching wells) = logit^{-1}(0.61 − 0.62 ∗* `dist100`*), with (jittered) data overlaid. The predictor* `dist100` *is* `dist`*/100: distance to the nearest safe well in 100-meter units.*

```
            Median MAD_SD
(Intercept)  0.606  0.059
dist        -0.006  0.001
```

The coefficient for `dist` is −0.006, which seems low, but this is misleading since distance is measured in meters, so this coefficient corresponds to the difference between, say, a house that is 90 meters away from the nearest safe well and a house that is 91 meters away.

Figure 13.8a shows the distribution of `dist` in the data. It seems more reasonable to rescale distance in 100-meter units:

```
wells$dist100 <- wells$dist/100
```

and re-fitting the logistic regression yields,

```
            Median MAD_SD
(Intercept)  0.6    0.1
dist100     -0.6    0.1
```

Graphing the fitted model

In preparing to plot the data, we first create a function to jitter the binary outcome while keeping the points between 0 and 1:

```
jitter_binary <- function(a, jitt=0.05){
  ifelse(a==0, runif(length(a), 0, jitt), runif(length(a), 1 - jitt, 1))
}
```

We can then graph the data and fitted model:

```
wells$switch_jitter <- jitter_binary(wells$switch)
plot(wells$dist, wells$switch_jitter)
curve(invlogit(coef(fit_1)[1] + coef(fit_1)[2]*x), add=TRUE)
```

The result is displayed in Figure 13.8b. The probability of switching is about 60% for people who live near a safe well, declining to about 20% for people who live more than 300 meters from any safe well. This makes sense: the probability of switching is higher for people who live closer to a safe well. Another display option, which would more clearly show the differences between households that did and did not switch, would be to overlay separate histograms of `dist` for the switchers and nonswitchers.

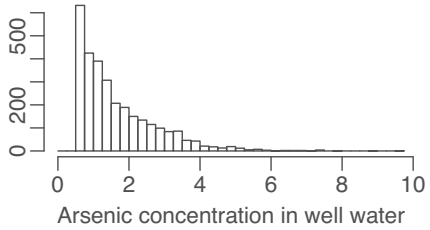


Figure 13.9 *Histogram of arsenic levels in unsafe wells (those exceeding 0.5) in the measured area of Araihazar, Bangladesh (see Figure 13.7).*

Interpreting the logistic regression coefficients

We can interpret the coefficient estimates using evaluations of the inverse logit function and its derivative, as in the example of Section 13.1. Our model here is,

$$\Pr(\text{switch}) = \text{logit}^{-1}(0.61 - 0.62 * \text{dist100}).$$

1. The constant term can be interpreted when `dist100` = 0, in which case the probability of switching is $\text{logit}^{-1}(0.61) = 0.65$. Thus, the model estimates a 65% probability of switching if you live right next to an existing safe well.
2. We can evaluate the predictive difference with respect to `dist100` by computing the derivative at the average value of `dist100` in the dataset, which is 0.48 (that is, 48 meters; see Figure 13.8a). The value of the linear predictor here is $0.61 - 0.62 * 0.48 = 0.31$, and so the slope of the curve at this point is $-0.62\, e^{0.31}/(1 + e^{0.31})^2 = -0.15$. Thus, adding 1 to `dist100`—that is, adding 100 meters to the distance to the nearest safe well—corresponds to a negative difference in the probability of switching of about 15%.
3. More quickly, the divide-by-4 rule gives us $-0.62/4 = -0.15$. This comes out the same, to two decimal places, as the result from calculating the derivative, because in this case the curve happens to pass through the 50% point right in the middle of the data; see Figure 13.8b.

In addition to interpreting its magnitude, we can look at the uncertainty of the coefficient for distance. The slope is estimated well, with a standard error of only 0.10, which is tiny compared to the coefficient estimate of −0.62. The approximate 95% interval is [−0.73, −0.49]. Overall probability of switching a well is 57% and the corresponding log score is −2059. The LOO log score for the model with distance is −2040; distance supplies some predictive information.

Adding a second input variable

We now extend the well-switching example by adding the arsenic level of the existing well as a regression input. At the levels present in the Bangladesh drinking water, the health risks from arsenic are roughly proportional to exposure, and so we would expect switching to be more likely from wells with high arsenic levels. Figure 13.9 shows the arsenic levels of the unsafe wells before switching, and here is the logistic regression predicting switching given arsenic level:

```
fit_3 <- stan_glm(switch ~ dist100 + arsenic, family=binomial(link="logit"), data=wells)
```

which, when displayed, yields,

```
            Median MAD_SD
(Intercept)  0.00   0.08
dist100     -0.90   0.11
arsenic      0.46   0.04
```

Thus, comparing two wells with the same arsenic level, every 100 meters in distance to the nearest safe well corresponds to a *negative* difference of 0.90 in the logit probability of switching. Similarly, a difference of 1 in arsenic concentration corresponds to a 0.46 *positive* difference in the logit probability of switching. Both coefficients are large compared to their standard errors. And both their signs make sense: switching is easier if there is a nearby safe well, and if a household's existing well has a high arsenic level, there should be more motivation to switch.

For a quick interpretation, we divide each of the coefficients by 4: thus, 100 meters more in distance corresponds to an approximately 22% lower probability of switching, and 1 unit more in arsenic concentration corresponds to an approximately 11% positive difference in switching probability.

Comparing these two coefficients, it would at first seem that distance is a more important factor than arsenic level in determining the probability of switching. Such a statement is misleading, however, because in our data `dist100` shows less variation than `arsenic`: the standard deviation of distances to the nearest well is 0.38 (in units of 100 meters), whereas arsenic levels have a standard deviation of 1.10 on the scale used here. Thus, the logistic regression coefficients corresponding to 1-standard-deviation differences are $-0.90 * 0.38 = -0.34$ for distance and $0.46 * 1.10 = 0.51$ for arsenic level. Dividing by 4 yields the quick summary estimate that a difference of 1 standard deviation in distance corresponds to an expected $-0.34/4$ or approximately 8% negative difference in Pr(switch), and a difference of 1 standard deviation in arsenic level corresponds to an expected $0.51/4$ or approximately 13% positive difference in Pr(switch).

The LOO log score for the new model is -1968.5, and comparison to the model with distance as the only predictor yields,

```
Model comparison:
(negative 'elpd_diff' favors 1st model, positive favors 2nd)
elpd_diff        se
     71.6      12.2
```

Including arsenic level in the model clearly improves the predictive accuracy.

The log score is good for model comparison as it takes properly into account the increase (or decrease) in the predictive information. It can be also useful to consider other summaries that are more directly interpretable. The average improvement in LOO predictive probabilities describes how much more accurately on average we are predicting the probabilities, although with the drawback that that probabilities close to zero can't improve much in absolute value even if they can improve a lot relatively.

Comparing the coefficient estimates when adding a predictor

The coefficient for `dist100` changes from -0.62 in the original model to -0.90 when arsenic level is added to the model. This change occurs because wells that are far from the nearest safe well are also likely to be particularly high in arsenic.

Graphing the fitted model with two predictors

The most natural way to graph the regression of y on two predictors might be as a three-dimensional surface, with the vertical axis showing Pr(y=1) as a function of predictors plotted on two horizontal axes.

However, we find such graphs hard to read, so instead we make separate plots as a function of each of the two variables; see Figure 13.10. As with the lines in Figure 10.4, we can plot the focus input variable on the x-axis and use multiple lines to show the fit for different values of the other input. To produce Figure 13.10a, we first plot the (jittered) data points, forcing zero to be included in the x-range of the plot because it is a natural baseline comparison for distance:

```
plot(wells$dist, wells$switch_jitter, xlim=c(0,max(wells$dist)))
```

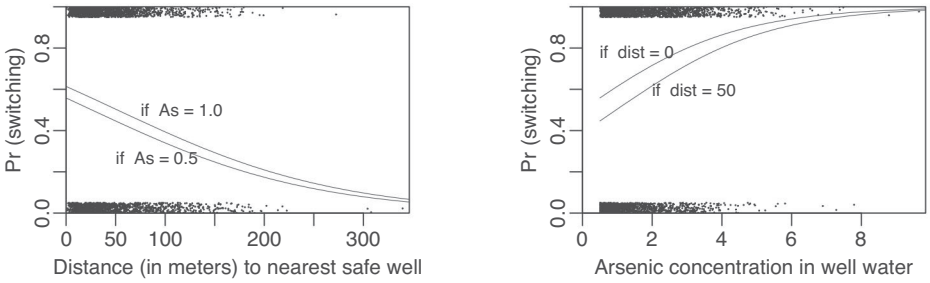


Figure 13.10 *Fitted logistic regression of probability of switching from an unsafe well as a function of two variables, plotted (a) as a function of distance to nearest safe well and (b) as a function of arsenic level of existing well. For each plot, the other input variable is held constant at different representative values.*

We next add the fitted curves:

```
curve(invlogit(cbind(1, x/100, 0.5) %*% coef(fit_3)), add=TRUE)
curve(invlogit(cbind(1, x/100, 1.0) %*% coef(fit_3)), add=TRUE)
```

We need to divide x by 100 here because the plot is in the scale of meters but the model is defined in terms of `dist100` = `dist`/100.

The object created by `cbind(1, x/100, 0.5)` is an $n \times 3$ matrix constructed from a column of 1's, the vector `x` (used internally by the `curve` function), and a vector of 0.5's. In constructing the matrix, R automatically expands the scalars 1 and 0.5 to the length of the vector `x`. For the two lines, we pick arsenic levels of 0.5 and 1.0 because 0.5 is the minimum value of arsenic concentration (since we are only studying users of unsafe wells), and a difference of 0.5 represents a reasonable comparison, given the distribution of arsenic levels in the data (see Figure 13.9).

Similar commands are used to make Figure 13.10b, showing the probability of switching as a function of arsenic concentration with distance held constant:

```
plot(wells$arsenic, wells$switch_jitter, xlim=c(0,max(wells$arsenic)))
curve(invlogit(cbind(1, 0, x) %*% coef(fit_3)), add=TRUE)
curve(invlogit(cbind(1,.5, x) %*% coef(fit_3)), add=TRUE)
```

13.8 Bibliographic note

According to Cramer (2003, chapter 9), logistic regression was introduced for binary data in the mid-twentieth century and has become increasingly popular as computational improvements have allowed it to become a routine data-analytic tool. McCullagh and Nelder (1989) discuss logistic regression in the context of generalized linear models, a topic we consider further in Chapter 15.

For more on income and voting in presidential elections, see Gelman, Park, et al. (2009). The example of drinking water in Bangladesh is described further by van Geen et al. (2003) and Gelman, Trevisani, et al. (2004).

13.9 Exercises

13.1 *Fitting logistic regression to data*: The folder `NES` contains the survey data of presidential preference and income for the 1992 election analyzed in Section 13.1, along with other variables including sex, ethnicity, education, party identification, and political ideology.

(a) Fit a logistic regression predicting support for Bush given all these inputs. Consider how to include these as regression predictors and also consider possible interactions.

(b) Evaluate and compare the different models you have fit.

(c) For your chosen model, discuss and compare the importance of each input variable in the prediction.

13.2 *Sketching the logistic curve*: Sketch the following logistic regression curves with pen on paper:

(a) $\Pr(y = 1) = \text{logit}^{-1}(x)$
(b) $\Pr(y = 1) = \text{logit}^{-1}(2 + x)$
(c) $\Pr(y = 1) = \text{logit}^{-1}(2x)$
(d) $\Pr(y = 1) = \text{logit}^{-1}(2 + 2x)$
(e) $\Pr(y = 1) = \text{logit}^{-1}(-2x)$

13.3 *Understanding logistic regression coefficients*: In Chapter 7 we fit a model predicting incumbent party's two-party vote percentage given economic growth: vote = 46.2 + 3.1 ∗ growth + error, where `growth` ranges from −0.5 to 4.5 in the data, and errors are approximately normally distributed with mean 0 and standard deviation 3.8. Suppose instead we were to fit a logistic regression, $\Pr(\text{vote} > 50) = \text{logit}^{-1}(a + b * \text{growth})$. Approximately what are the estimates of (a, b)?

Figure this out in four steps: (i) use the fitted linear regression model to estimate $\Pr(\text{vote} > 50)$ for different values of `growth`; (ii) second, plot these probabilities and draw a logistic curve through them; (iii) use the divide-by-4 rule to estimate the slope of the logistic regression model; (iv) use the point where the probability goes through 0.5 to deduce the intercept. Do all this using the above information, without downloading the data and fitting the model.

13.4 *Logistic regression with two predictors*: The following logistic regression has been fit:

```
            Median MAD_SD
(Intercept) -1.9    0.6
x            0.7    0.8
z            0.7    0.5
```

Here, x is a continuous predictor ranging from 0 to 10, and z is a binary predictor taking on the values 0 and 1. Display the fitted model as two curves on a graph of $\Pr(y = 1)$ vs. x.

13.5 *Interpreting logistic regression coefficients*: Here is a fitted model from the Bangladesh analysis predicting whether a person with high-arsenic drinking water will switch wells, given the arsenic level in their existing well and the distance to the nearest safe well:

```
stan_glm(formula = switch ~ dist100 + arsenic, family=binomial(link="logit"),
  data=wells)
             Median MAD_SD
(Intercept)   0.00   0.08
dist100      -0.90   0.10
arsenic       0.46   0.04
```

Compare two people who live the same distance from the nearest well but whose arsenic levels differ, with one person having an arsenic level of 0.5 and the other person having a level of 1.0. You will estimate how much more likely this second person is to switch wells. Give an approximate estimate, standard error, 50% interval, and 95% interval, using two different methods:

(a) Use the divide-by-4 rule, based on the information from this regression output.
(b) Use predictive simulation from the fitted model in R, under the assumption that these two people each live 50 meters from the nearest safe well.

13.6 *Interpreting logistic regression coefficient uncertainties*: In Section 13.7, there were two models, `fit_4` and `fit_5`, with distance and arsenic levels as predictors along with an interaction term. The model `fit_5` differed by using centered predictors. Compare the reported uncertainty estimates (mad sd) for the coefficients, and use for example the `mcmc_pairs` function in the `bayesplot` package to examine the pairwise joint posterior distributions. Explain why the mad sd values are different for `fit_4` and `fit_5`.

13.7 *Graphing a fitted logistic regression*: We downloaded data with weight (in pounds) and age (in years) from a random sample of American adults. We then defined a new variable:

```
heavy <- weight > 200
```

and fit a logistic regression, predicting `heavy` from `height` (in inches):

```
stan_glm(formula = heavy ~ height, family=binomial(link="logit"), data=health)
             Median MAD_SD
(Intercept)  -21.51   1.60
height         0.28   0.02
```

(a) Graph the logistic regression curve (the probability that someone is heavy) over the approximate range of the data. Be clear where the line goes through the 50% probability point.

(b) Fill in the blank: near the 50% point, comparing two people who differ by one inch in height, you'll expect a difference of ____ in the probability of being heavy.

13.8 *Linear transformations*: In the regression from the previous exercise, suppose you replaced height in inches by height in centimeters. What would then be the intercept and slope?

13.9 *The algebra of logistic regression with one predictor*: You are interested in how well the combined earnings of the parents in a child's family predicts high school graduation. You are told that the probability a child graduates from high school is 27% for children whose parents earn no income and is 88% for children whose parents earn $60 000. Determine the logistic regression model that is consistent with this information. For simplicity, you may want to assume that income is measured in units of $10 000.

13.10 *Expressing a comparison of proportions as a logistic regression*: A randomized experiment is performed within a survey, and 1000 people are contacted. Half the people contacted are promised a $5 incentive to participate, and half are not promised an incentive. The result is a 50% response rate among the treated group and 40% response rate among the control group.

(a) Set up these results as data in R. From these data, fit a logistic regression of response on the treatment indicator.

(b) Compare to the results from Exercise 4.1.

13.11 *Building a logistic regression model*: The folder `Rodents` contains data on rodents in a sample of New York City apartments.

(a) Build a logistic regression model to predict the presence of rodents (the variable `rodent2` in the dataset) given indicators for the ethnic groups (`race`). Combine categories as appropriate. Discuss the estimated coefficients in the model.

(b) Add to your model some other potentially relevant predictors describing the apartment, building, and community district. Build your model using the general principles explained in Section 12.6. Discuss the coefficients for the ethnicity indicators in your model.

13.12 *Fake-data simulation to evaluate a statistical procedure*: When can we get away with fitting linear regression to binary data? You will explore this question by simulating data from a logistic regression, then fitting a linear regression, then looping this procedure to compute the coverage of the estimates.

(a) You will be simulating independent binary data, $y_i, i = 1, \ldots, n$, from the model, $\Pr(y_i = 1) = \text{logit}^{-1}(a + bx_i + \theta z_i)$, where the x_i's are drawn uniformly from the range $(0, 100)$ and the z_i's are randomly set to 0 or 1. The "cover story" here is that y represents passing or failing an exam, x is the score on a pre-test, and z is a treatment.

To do this simulation, you will need to set true values of a, b, and θ. Choose a and b so that 60% of the students in the control group will pass the exam, with the probability of passing being 80% for students in the control group who scored 100 on the midterm.

Choose θ so that the average probability of passing increases by 10 percentage points under the treatment.

Report your values for a, b, θ and explain your reasoning (including simulation code). It's not enough just to guess.

(b) Simulate $n = 50$ data points from your model, and then fit a linear regression of y on x and z. Look at the estimate and standard error for the coefficient of z. Does the true average treatment effect fall inside this interval?

(c) Repeat your simulation in (b) 10 000 times. Compute the coverage of the normal-theory 50% and 95% intervals (that is, the estimates $\pm$ 0.67 and 1.96 standard errors).

13.13 *Working through your own example*: Continuing the example from the final exercises of the earlier chapters, fit a logistic regression, graph the data and fitted model, and interpret the estimated parameters and their uncertainties.